



Documentation haut-niveau du prototype Dora

Auteurs :
Aymen ALI YAHIA, Mohammed DOUIDY

3 août 2025

Table des matières

1	Introduction	3
1.1	Prototype Dora	3
1.2	Setup matériel	3
2	Eye-tracking	5
2.1	Présentation générale	5
2.2	Installation et paramétrage de Pupil Capture	5
2.3	Bonnes pratiques	6
2.4	Pupil Capture dans Unity	8
2.4.1	Architecture générale du système	8
2.4.2	Calibration secondaire dans Unity	9
3	Head-tracking	10
3.1	Présentation générale	10
3.2	Configuration et installation	10
3.2.1	Composants nécessaires	10
3.2.2	Reconstruction depuis les sources	11
3.3	Paramétrage dans Opentrack	12
3.4	Opentrack dans Unity	13
3.4.1	Architecture générale du système	13
3.4.2	UDPReceiver	14
3.4.3	HeadPositionController	15
3.4.4	CameraController	15
4	Cécité au changement	17
4.1	Présentation générale	17
4.2	Architecture du système	17
4.2.1	Composants principaux	17
4.2.2	Configuration du ChangeBlindnessManager	17
4.3	Exemples d'expériences disponibles	18
4.3.1	Expériences InFoV	18
4.3.2	Expériences OutFoV	19
4.4	Ajout d'expériences personnalisées	19
4.4.1	Création d'une nouvelle expérience	19
4.4.2	Méthodes à implémenter	19
4.4.3	Exemple d'implémentation	20
4.5	Gestion et intégration	21
4.5.1	Gestion de l'état	21
4.5.2	Intégration avec l'eye-tracking	21

5	Système d'illusions visuelles	22
5.1	Présentation générale	22
5.2	Architecture du système	22
5.2.1	Composants principaux	22
5.2.2	Configuration de l'EffectsManager	22
5.2.3	Configuration du PostProcessManager	23
5.3	Effets disponibles	24
5.3.1	Profondeur de champ (DepthOfField)	24
5.3.2	Vignettage dynamique (Vignette)	24
5.3.3	Aberration chromatique (ChromaticAberration)	24
5.3.4	Effet de surbrillance (Bloom)	24
5.3.5	Distortion de lentille (LensDistortion)	24
5.4	Interface utilisateur	25
5.4.1	Configuration de l'IllusionUI	25
5.5	Ajout d'effets personnalisés	25
5.5.1	Création d'un nouvel effet	25
5.5.2	Implémentation d'une méthode de mise à jour	25
5.5.3	Intégration dans le système	25
6	Analyseur de zones de regard (GazeAnalyzer)	26
6.1	Présentation générale	26
6.2	Architecture du système	26
6.2.1	Composants principaux	26
6.2.2	Configuration du GazeZoneAnalyzer	26
6.2.3	Configuration du GazeProportionAnalyzer	27
6.3	Fonctionnement technique	27
6.3.1	Principe de segmentation par identifiants colorés	27
6.3.2	Analyse de la zone de regard	28
6.3.3	Traitement asynchrone	28
6.3.4	Interprétation des résultats	28
7	Système de search light	29
7.1	Présentation générale	29
7.2	Changement des propriétés des matériaux	29
7.3	Profondeur minimum et maximum	30

1. Introduction

1.1 Prototype Dora

Ce prototype a pour but de simplifier la mise en place d'expériences permettant l'exploration de la perception d'un utilisateur à l'aide plusieurs modifications dans la scène, hors du champ de vision ainsi qu'à l'intérieur. Le prototype est un prefab, portable sur n'importe quelle scène Unity créée avec la pipeline URP. Le prefab est situé dans le répertoire `Assets/Prefabs/tracking` nommé `Tracking System`. Ce dernier est constitué de deux parties, une partie pour les gestionnaires et une autre pour les éléments de visualisation.

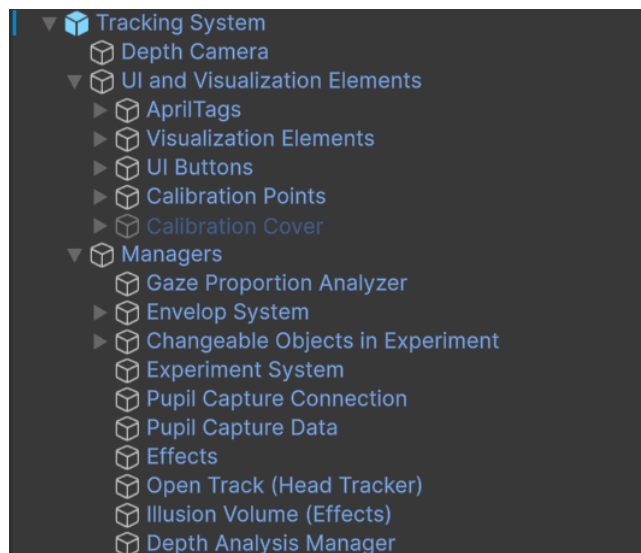


FIGURE 1.1 – Illustration du prefab Unity

Pour les gestionnaires, chaque `GameObject` est associé à un ou plusieurs scripts, qui nous permettent de paramétrer les différents gestionnaires afin de modifier les expériences comme voulu. Pour les éléments de visualisations, ils sont hiérarchisés de façon à séparer le interface utilisateur et les autres éléments de visualisations, qui peuvent être modifiés.

1.2 Setup matériel

La configuration optimale est de connecter les différents trackers sur une même machine, cette dernière va envoyer les données des trackers à la machine qui exécute la scène Unity. Cette configuration permet d'alléger la charge sur le CPU de la machine qui exécute la scène Unity, permettant d'avoir le moins de latence possible.

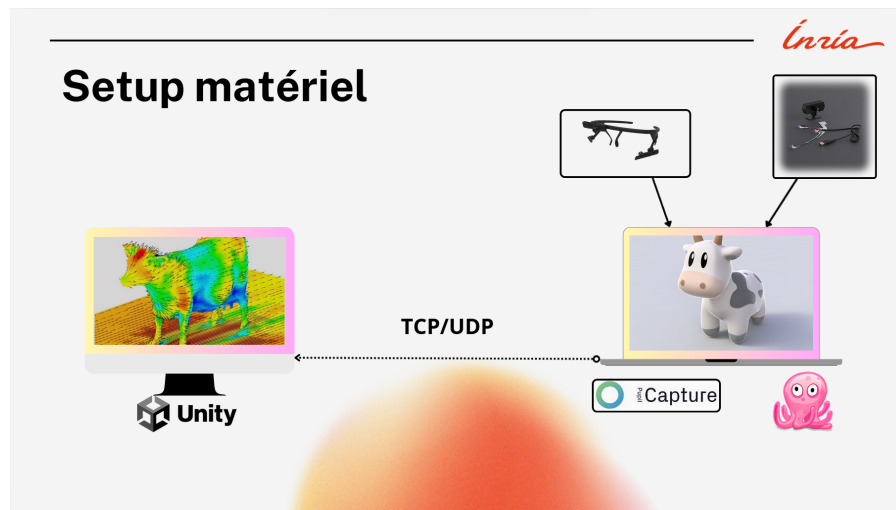


FIGURE 1.2 – Illustration du setup matériel

Cependant, il est possible de lancer tous les programmes sur une même machine, à condition que le CPU soit assez performant et que la machine ait environ 24 GB de RAM.

2. Eye-tracking

2.1 Présentation générale

Le système de eye-tracking permet à l'utilisateur modifier dynamiquement la scène 3D en bougeant le regard. Il utilise deux cameras infrarouge et une camera monde avec des marqueurs placés sur la scène Unity afin d'avoir la position du regard dans la scène en temps réel. La position parmi, d'autres informations, est envoyée via le réseau. Ces informations permettent de réaliser un certain nombre de calculs permettant d'obtenir les objets contenus dans la zone de regard, des informations de profondeurs, etc...

2.2 Installation et paramétrage de Pupil Capture

Sachant qu'une version modifiée de Pupil Capture est nécessaire pour faire fonctionner ce prototype, voici les étapes à suivre afin de compiler et d'exécuter le programme :

- Installer et télécharger Python 3.11.
- Lancer Powershell en admin au niveau du répertoire Dora/pupil.
- Créer un environnement virtuel avec Python 3.11 via la commande :

```
python -3.11 -m venv venv
```

- Lancez l'environnement virtuel via la commande :

```
.\venv\Scripts\activate
```

- Installer les dépendances via les commande :

```
python -m pip install --upgrade pip setuptools  
python -m pip install -r requirements.txt
```

- Lancer le programme dans le répertoire pupil_src via la commande :

```
.\pupil\pupil_src\main.py capture
```

- Si jamais la version de NumPy pose problème, effectuer les commandes suivantes :

```
pip uninstall numpy  
pip install "numpy<2.0"
```

Certains paramètres de Pupil Capture influent sur les performances (latence et justesse) du eye-tracker. Voici quelques paramètres à modifier pour avoir de meilleures performances :

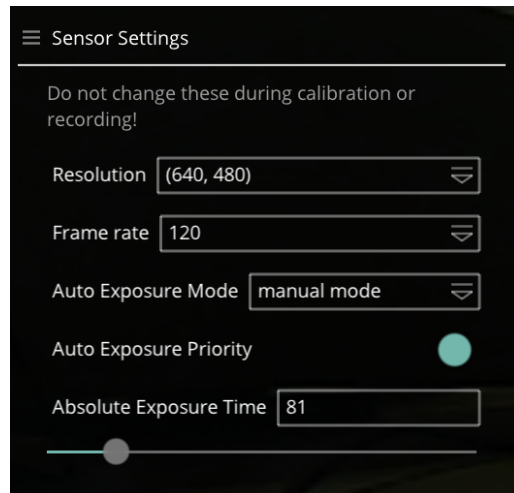


FIGURE 2.1 – Paramètres à utiliser pour la camera monde

Pour la camera monde, les paramètres les plus importants sont la résolution et l'exposition. On trouve que la résolution 640x480 est le bon compromis entre détection stable des tags et faible latence. Pour améliorer la détection des tags et aussi la latence, on diminue le **Absolute Exposure Time** autant que possible (en ayant **manual mode** sélectionné dans le champ **Auto Exposure Mode**). On vérifie la stabilité de détection des tags à la stabilité de la surface détecté autour des tags.

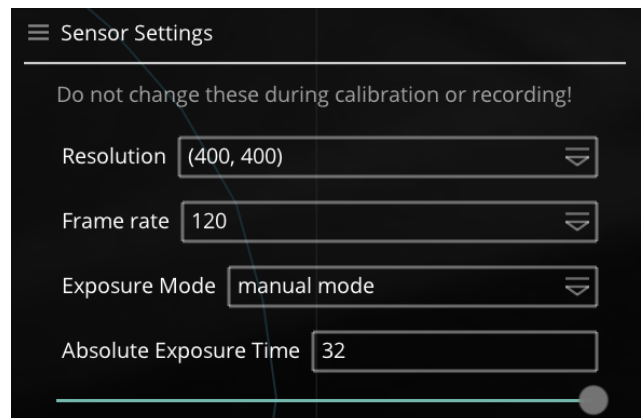


FIGURE 2.2 – Paramètres à utiliser pour les cameras des yeux

Pour les cameras des yeux, le paramètre principal est l'exposition. Il faut diminuer ce dernier (en ayant **manual mode** sélectionné dans le champ **Exposure Mode**) pour avoir une détection stable des pupilles et une bonne latence. La résolution n'impacte que très peu la détection des pupilles, on choisit donc la plus haute résolution disponible.

2.3 Bonnes pratiques

Pour garantir une bonne qualité de suivi oculaire, il est recommandé d'utiliser la pipeline 3D lorsqu'une robustesse face aux mouvements est requise, et la pipeline 2D lorsque la priorité est la justesse de la détection. Il faut aussi de désactiver tous les plugins non utilisés pour limiter les interférences et améliorer la stabilité du système.

Avant toute calibration, plusieurs réglages sont nécessaires pour assurer une détection optimale. Il est essentiel de bien centrer l'œil à l'aide des molettes et des extensions du capteur, d'ajuster le ROI (à activer dans chaque fenêtre de retour des caméras infrarouges) ainsi que la zone de détection de la pupille, en faisant des tours avec les yeux, afin qu'elles soient aussi réduites que possible tout en englobant l'ensemble des mouvements de l'œil. La taille des tags utilisés dans la scène doit également être adaptée pour garantir une détection stable, dépendant la taille de l'écran utilisé pendant les expériences. Le focus de la caméra monde doit être précisément ajusté afin d'optimiser la détection des tags, et l'exposition doit être réglée manuellement : pour la caméra monde, pour stabiliser la détection des tags, et pour les caméras oculaires, afin d'atteindre une confiance proche de 1.

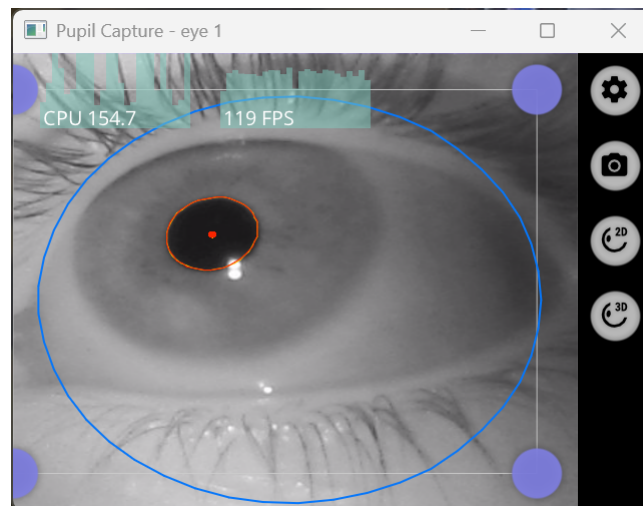


FIGURE 2.3 – Illustration de la zone d'intérêt (rectangle en bleu) et la zone de mouvement de l'œil (ellipse en bleu)

Certaines conditions physiologiques peuvent influencer la qualité de la détection. Il est conseillé d'éviter le maquillage, en particulier le mascara, ainsi que des cils trop fournis, car ceux-ci perturbent la détection des pupilles. De plus, les yeux de couleur claire sont généralement moins bien perçus par les algorithmes, ce qui peut affecter la fiabilité des mesures.

La phase de calibration est très importante, étant le facteur de variabilité le plus impactant. Il est recommandé d'utiliser une calibration à 9 points pour de meilleurs résultats. Durant cette phase, l'utilisateur doit rester aussi neutre et stable que possible pour assurer une correspondance fiable entre les points de calibration et les données recueillies. La calibration peut se faire sur chacune des deux machines, aucune différence n'a été remarqué, à condition que la distance entre l'utilisateur et la machine de calibration soit similaire à la distance entre l'utilisateur et la machine pour les expériences.

Enfin, pendant toute la durée de l'expérience, il est crucial d'éviter les mouvements brusques qui peuvent altérer significativement la précision du suivi oculaire à cause des mouvements du eye-tracker sur le visage. Si la justesse du regard n'est pas satisfaisante, il ne faut pas hésiter à relancer plusieurs calibrations dans le logiciel Capture jusqu'à obtenir un résultat optimal, qui est autour des 1 degré.

Pour plus d'information sur les bonnes pratiques, [les recommandations du constructeur](#)

2.4 Pupil Capture dans Unity

2.4.1 Architecture générale du système

Connexion de Pupil Capture dans Unity

Pour lancer la scène avec le eye-tracking, il faut remplir le champ **Pupil Capture Connection** avec l'adresse IP situé dans l'onglet connexion de Pupil Capture. Choisir la bonne adresse par rapport au mode de connexion utilisé.

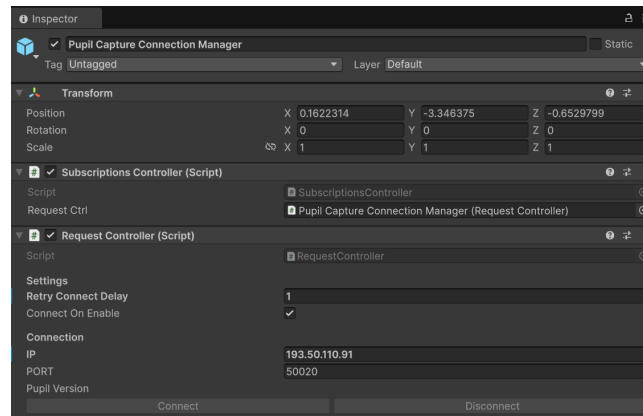


FIGURE 2.4 – Connexion de Pupil Capture avec Unity

Configuration de Pupil Capture Data

Ensuite, il faut remplir tous les champs de **Pupil Capture Data** (Gaze Manager) comme suit :

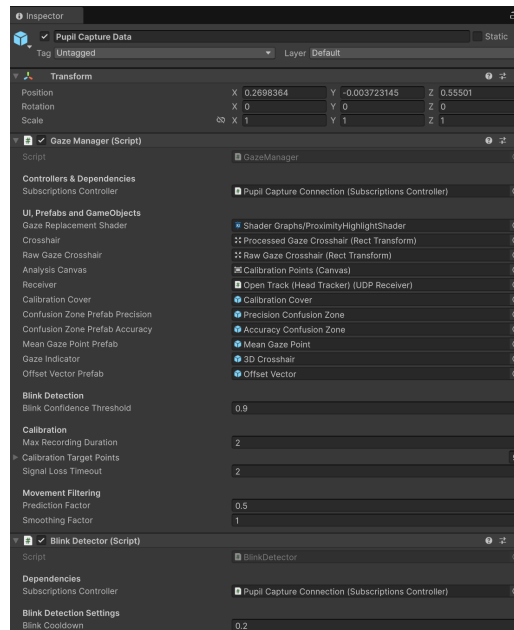


FIGURE 2.5 – Paramètres de Pupil Capture Data à utiliser

Dans la section **UI, Prefabs and GameObjects**, il suffit de suivre la même configuration. Cependant, les autres champs où il y a des valeurs à modifier, ces dernières peuvent être

modifiée. Lire la documentation technique pour plus d'information.

2.4.2 Calibration secondaire dans Unity

Si les résultats de la calibration dans Pupil Capture ne sont pas satisfaisant, alors une calibration secondaire est possible dans **Unity**. Cette calibration se passe en plusieurs étapes :

1. Initialisation de la calibration : Appuyer sur le bouton **Calibration** dans la scène afin d'afficher un écran noir pour diminuer les distractions visuelles ;
2. Appuyer sur la touche **S** : Pour afficher la première cible de calibration, la calibration n'a pas encore commencé ;
3. Une fois que le regard est fixé sur la cible, appuyer sur la touche **R** : afin d'enregistrer les positions du regard par rapport à la cible en question : application de l'effet fenêtre virtuelle.
4. Il faut refaire ces deux précédentes étapes sur les neufs points de calibration : une fois les neufs points effectué, la calibration est terminée.
5. Appuyer sur la touche **N** pour appliquer la calibration et afficher les résultats à l'écran : on voit les vecteurs d'offset à chaque point de calibration et la zone de confusion qui change par rapport à la calibration.
6. Pour enlever la visualisation des résultats de la calibration, appuyer sur la touche **V** : après cela, vous pouvez refaire une calibration en appuyant sur la touche **S** et refaire les étapes, ou bien revenir sur la scène en appuyant sur le bouton **Calibration**.

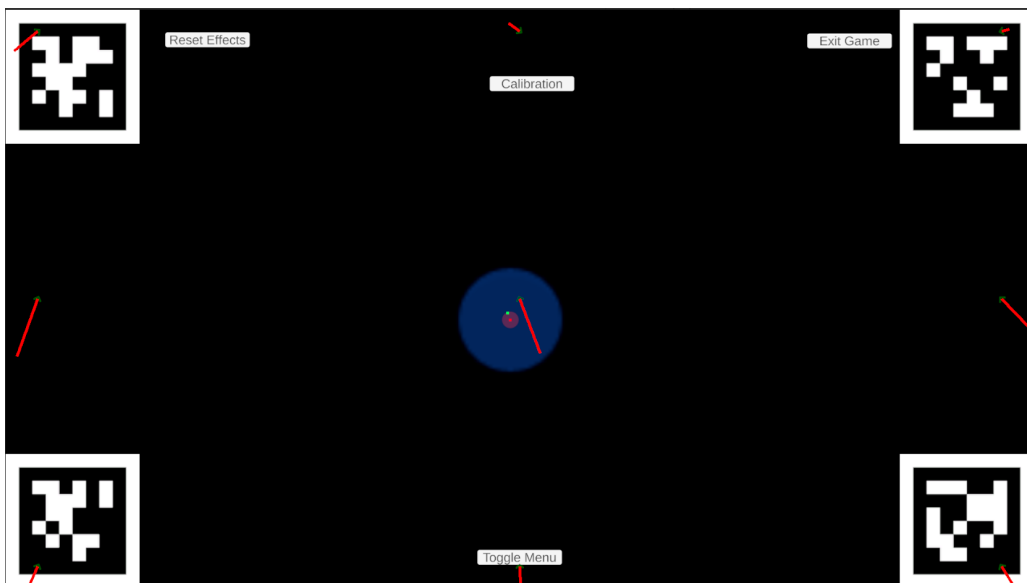


FIGURE 2.6 – Illustration des résultats de la calibration dans Unity

3. Head-tracking

3.1 Présentation générale

Le système de head-tracking permet à l'utilisateur d'explorer dynamiquement la scène 3D en bougeant sa tête. Il utilise un dispositif infrarouge avec des marqueurs placés sur un support métallique pouvant être attaché à un casque porté par l'utilisateur pour capturer les mouvements en temps réel. Ces mouvements sont envoyés via le réseau sous forme de position et, comme la version d'OpenTrack utilisée est modifiée, le nombre de points infrarouges couramment capturés est aussi récupérable. Ces informations permettent de réaliser un certain nombre de calculs permettant d'obtenir un effet de fenêtre physique sur un monde virtuel.

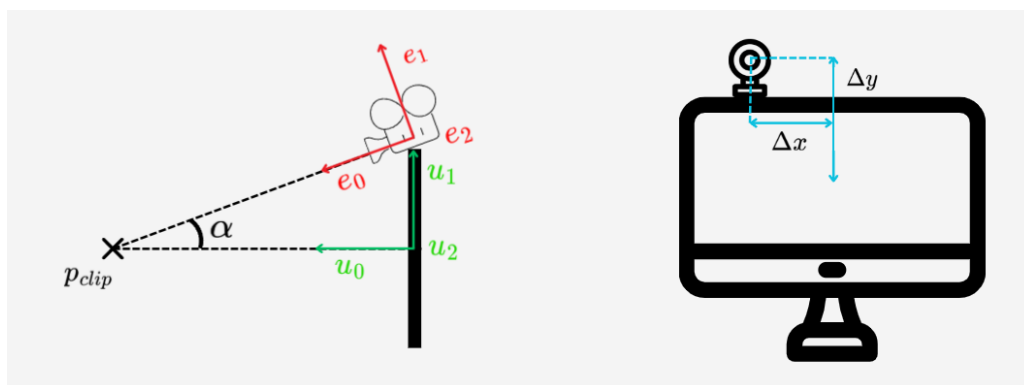


FIGURE 3.1 – Schéma du système de head-tracking avec dispositif infrarouge pris d’une diapositive de la présentation du prototype à l’équipe Manao. A gauche, un schéma illustrant la transformation nécessaire pour passer de la base caméra (rouge) à la base centre de l’écran (verte). Elle nécessite de connaître l’angle alpha et les offsets x et y en centimètres (vus sur le schéma de droite) qui peuvent être mis dans l’inspecteur pour qu’elle soit appliquée dans le code en recevant les données.

3.2 Configuration et installation

3.2.1 Composants nécessaires

- OpenTrack dont une version modifiée est fournie dans le dossier du projet. Il est situé dans `Opentrack/build/install/opentrack.exe` ;
- caméra infrarouge ;
- leds infrarouges à placer sur un casque ou support.

3.2.2 Reconstruction depuis les sources

Cette partie explique comment reconstruire OpenTrack à partir du code source inclus avec le projet si des modifications venaient à être effectuées à ce dernier.

Prérequis

- MSYS2 (64-bit) avec la chaîne d'outils MinGW-w64. Le package mingw-w64-x86_64-qt5 est également nécessaire et peut être installé avec l'invité de commande MSYS2 MinGW 64-bit via la commande :

```
pacman -S mingw-w64-x86_64-qt5
```

- OpenCV compatible avec la chaîne d'outils MinGW. Un dossier contenant le projet compilé pour la suite MinGW est disponible sur GitHub via <https://github.com/huihut/OpenCV-MinGW-Build>;
- CMake et CMake GUI.

Il faut ajouter C:\msys64\mingw64\bin au PATH système pour accéder aux outils Qt et mingw32-make.

Configuration CMake

- configuration dans CMake GUI :
 - source : chemin vers le dossier OpenTrack du projet ;
 - build : chemin vers le dossier OpenTrack_src/build du projet ;
 - générateur : "MinGW Makefiles".
- variables essentielles :
 - CMAKE_MAKE_PROGRAM → C:/msys64/mingw64/bin/mingw32-make.exe ;
 - OpenCV_DIR → dossier contenant OpenCVConfig.cmake ;
 - WITH_OPENCV = ON ;
 - OPENTRACK_PLUGIN_INPUT_PT = ON.

Name	Value
CMAKE_BUILD_TYPE	RELEASE
CMAKE_GNUtoMS	☐
CMAKE_INSTALL_PREFIX	C:/Users/douid/Desktop/Internship workspace/EXPLORA prototype/Opentrack_src/build/install
ONNXRuntime_DIR	
OPENTRACK_PLUGIN_INPUT_PT	☒
OpenCV_DIR	C:\Users\douid\Downloads\OpenCV-MinGW-Build-OpenCV-4.5.5-x64\OpenCV-MinGW-Build-OpenCV-4.5.5-x64\mingw\lib
Qt5Core_DIR	C:/msys64/mingw64/lib/cmake/Qt5Core
Qt5Gui_DIR	C:/msys64/mingw64/lib/cmake/Qt5Gui
Qt5LinguistTools_DIR	C:/msys64/mingw64/lib/cmake/Qt5LinguistTools
Qt5Network_DIR	C:/msys64/mingw64/lib/cmake/Qt5Network
Qt5SerialPort_DIR	C:/msys64/mingw64/lib/cmake/Qt5SerialPort
Qt5Widgets_DIR	C:/msys64/mingw64/lib/cmake/Qt5Widgets
Qt5_DIR	C:/msys64/mingw64/lib/cmake/Qt5
SDK_ARUCO_LIBPATH	
SDK_EYEWARE_BEAM	
SDK_HYDRA	
SDK_KINECT20	
SDK_OSCPACK	
SDK_REALSENSE	
SDK_RIFT_140	
SDK_TOBI	
SDK_TRACKHAT_SENSOR	
SDK_VALVE_STEAMVR	
SDK_VJOYSTICK	
WITH_OPENCV	☒

FIGURE 3.2 – Variables de compilation nécessaires dans CMake GUI. La plupart d'entre elles sont automatiquement prises en compte, mais certaines, décrites précédemment, doivent être manuellement créées.

- commande de build et d'installation à exécuter dans un invité de commande depuis le dossier /build **APRÈS** avoir généré le projet en appuyant sur **Generate** en bas de la fenêtre d'application de **CMake GUI** :

```
cmake --build . --target install
```

L'installation génère dans `Opentrack/build/install/` l'exécutable et les DLL nécessaires à son fonctionnement.

3.3 Paramétrage dans Opentrack

Le paramétrage dans Opentrack se fait en premier lieu avec le profil en `.ini` fourni par les constructeurs de la caméra et du support à leds infrarouges. Ce profil, sous la forme d'un fichier nommé **Delanclip Starter - Metal Clip**, est fourni avec le projet dans le dossier `Opentrack/` et doit être sélectionné dans le champ *Profile* de l'application. Les deux autres champs primordiaux sont :

- *Input* : dans lequel on choisit le plugin **PointTracker 1.1**.
- *Output* : dans lequel on choisit **UDP over network**. C'est la méthode chargée d'envoyer sur les réseau les paquets devant être réceptionnés par **Unity**. En cliquant sur le marteau à droite de la méthode, il est possible de choisir l'adresse IPv4 à laquelle envoyer les paquets ainsi que le port. Si Opentrack et **Unity** sont exécutés sur le même ordinateur, alors il suffit de mettre l'adresse de *loopback* 127.0.0.1. Sinon, il faut que l'ordinateur se chargeant des données et celui faisant tourner **Unity** soit sur le même réseau pour éviter les problématiques de pare-feu.

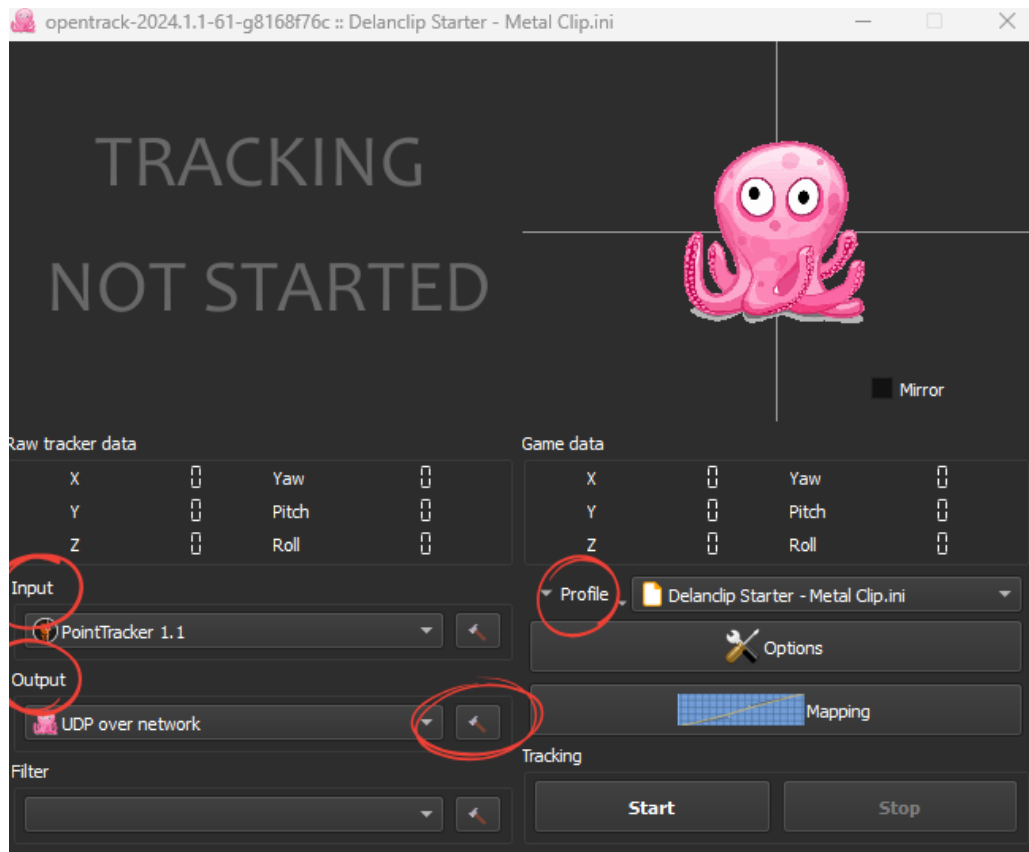


FIGURE 3.3 – Fenêtre Opentrack sur laquelle sont entourés les champs de paramétrage évoqués.

Comme on récupère les données brutes, dont la position donnée en centimètres par rapport à la caméra, et qu'on applique un léger filtrage de bruit par interpolation linéaire dans **Unity**, le champ *Filter* ne sélectionne aucune méthode de filtrage. Par ailleurs, l'avatar de visualisation de la position et de la rotation de la tête par rapport à la caméra ne fonctionne pas dans cette version modifiée du logiciel, car la nature des données en sortie est différente de la version originale.

3.4 Opentrack dans Unity

Le système de head-tracking dans **Unity** repose sur trois composants principaux qui fonctionnent ensemble pour créer l'effet de fenêtre virtuelle.

3.4.1 Architecture générale du système

Le flux de données suit cette séquence (en utilisant le nom des classes appropriées) :

1. **UDPReceiver** : réception et transformation des données brutes d'Opentrack ;
2. **HeadPositionController** : lissage et continuation de mouvement ;
3. **CameraController** : application de l'effet fenêtre virtuelle.

3.4.2 UDPReceiver

GameObject et rôle

L'UDPReceiver est attaché à un GameObject vide nommé *Open Track (Head Tracker)*. Ce composant constitue le point d'entrée des données du head-tracking dans Unity.

Paramètres configurables dans l'inspecteur

- **Transformation Settings :**
 - **Scale Factor** : facteur d'échelle appliqué aux coordonnées reçues (par défaut : 0.01f);
 - **X Rotation Angle Deg** : angle de rotation en degrés autour de l'axe X pour l'alignement des coordonnées (par défaut : 0.0f);
 - **Translation** : offsets de translation pour appliquer une transformation de la base caméra à la base centre de l'écran vue sur la figure 3.1 (par défaut : Vector2.zero).
- **Network Settings :**
 - **Port** : port UDP pour la réception des données (par défaut : 4242). **Cependant ce paramètre ne fonctionne pas car le port est noté à la main dans la méthode chargée de la connection à la suite d'erreurs répétées.**

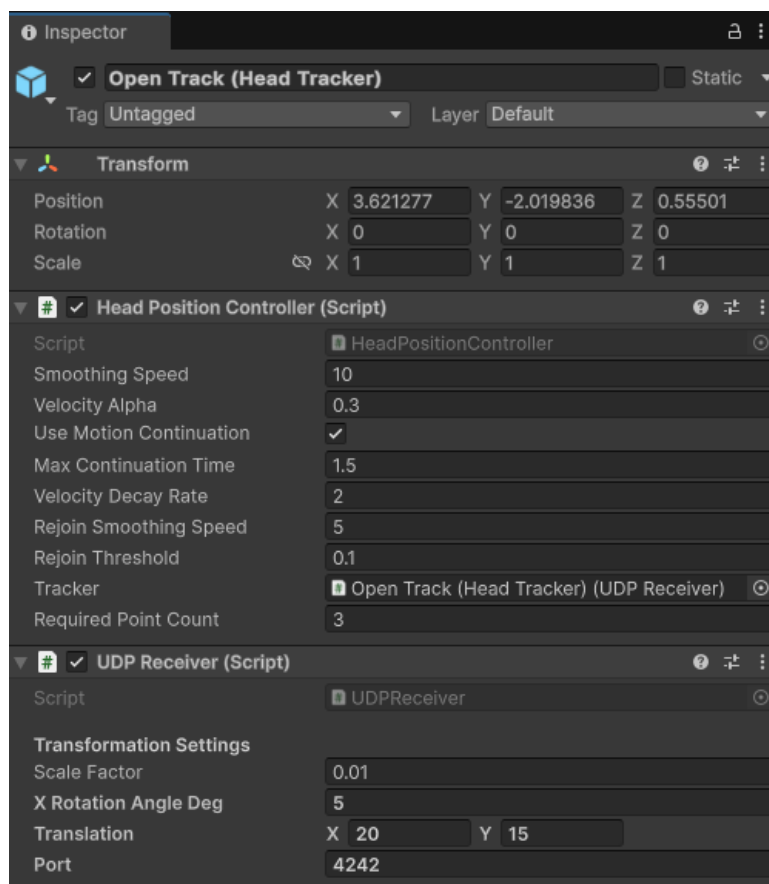


FIGURE 3.4 – Configuration du UDPReceiver et HeadPositionController dans l'inspecteur Unity.

3.4.3 HeadPositionController

GameObject et rôle

Le `HeadPositionController` est attaché au même `GameObject` que `UDPReceiver`. Il fournit le lissage des données et la continuité de mouvement en cas de perte de signal.

Paramètres configurables dans l'inspecteur

- **Smoothing Settings :**
 - **Smoothing Speed** : vitesse d'interpolation de position pendant le suivi normal (par défaut : 10f);
 - **Velocity Alpha** : facteur de lissage exponentiel pour l'estimation de vitesse (0-1, par défaut : 0.3f).
- **Motion Continuation :**
 - **Use Motion Continuation** : active la continuation prédictive lors de la perte de signal (par défaut : true);
 - **Max Continuation Time** : durée maximale de prédiction de mouvement (par défaut : 1.5f secondes);
 - **Velocity Decay Rate** : taux de décroissance exponentielle de la vitesse (par défaut : 2f).
- **Rejoining Settings :**
 - **Rejoin Smoothing Speed** : vitesse de lissage lors de la reconnexion (par défaut : 5f);
 - **Rejoin Threshold** : seuil de distance pour sortir du mode reconnexion (par défaut : 0.1f).
- **Tracker Settings :**
 - **Tracker** : référence vers le composant `UDPReceiver`;
 - **Required Point Count** : nombre minimum de points requis pour un suivi valide (par défaut : 3).

3.4.4 CameraController

GameObject et rôle

Le `CameraController` doit être attaché au `GameObject` de la caméra principale (*Main Camera*). Il applique l'effet de fenêtre virtuelle en ajustant dynamiquement la position, l'orientation et le champ de vision de la caméra.

Paramètres configurables

- **Head Tracking Reference :**
 - **Head Position** : référence vers le composant `HeadPositionController` qui fournit les données de position lissées

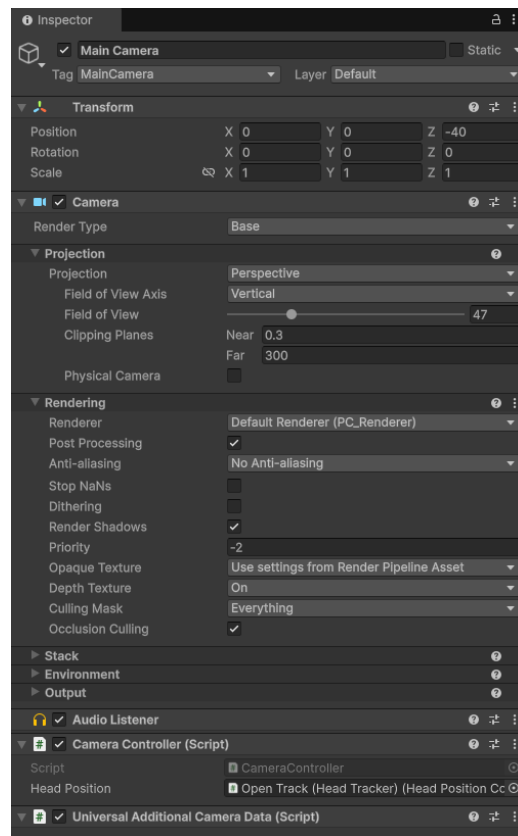


FIGURE 3.5 – Configuration du CameraController dans l’inspecteur Unity

4. Cécité au changement

4.1 Présentation générale

Le système de cécité au changement (ou *change blindness*) permet de réaliser des expériences de cécité au changement dans l'environnement virtuel **Unity**. Ce phénomène psychologique consiste en l'incapacité des observateurs à détecter des changements visuels significatifs lorsque ceux-ci se produisent pendant une interruption visuelle ou en dehors du champ de vision direct. Le système implémenté supporte deux types d'expériences principales (correspondant à une énumération dans le code, détaillée dans la documentation technique du projet) et est en partie basé sur les travaux de *Martin et al.* [1] :

- **InFoV** (*In Field of View*) : les changements se produisent lorsque l'objet est dans le champ de vision et que le regard de l'utilisateur est dirigé vers l'objet cible, généralement pendant un clignement d'œil pour masquer le changement ;
- **OutFoV** (*Out of Field of View*) : les changements se produisent lorsque l'objet est en dehors du champ de vision de l'utilisateur, créant une surprise lorsque celui-ci reporte son attention sur la zone modifiée.

4.2 Architecture du système

4.2.1 Composants principaux

Le système de change blindness est organisé autour de plusieurs composants clés :

- **ChangeBlindnessManager** : classe implémentant un gestionnaire central qui coordonne toutes les expériences et déclenche les changements selon les conditions définies ;
- **IExperiment** : interface définissant les capacités requises pour qu'un objet puisse participer aux expériences ;
- **BaseExperiment** : classe abstraite de base fournissant les fonctionnalités communes à tous les types d'expériences ;
- des expériences spécialisées : classes implémentant des types de changements spécifiques (couleur, position, échange d'objets, etc.).

4.2.2 Configuration du **ChangeBlindnessManager**

Le **ChangeBlindnessManager** constitue le cœur du système et doit être configuré dans l'inspecteur **Unity** avec les paramètres suivants :

- **Main Camera** : référence vers la caméra principale utilisée pour les calculs de champ de vision ;
- **Experiments** : liste de toutes les expériences (correspondant à un objet **Unity** vide auquel est attaché le script associé à l'expérience voulue) qui pourront être déclenchées lors du lancement de la scène ;
- **Min Time Between Changes** : intervalle minimum (en secondes) devant s'écouler entre deux changements consécutifs ;
- **Start Experiment Button** et **Reset Button** : boutons UI pour le contrôle de l'expérience.

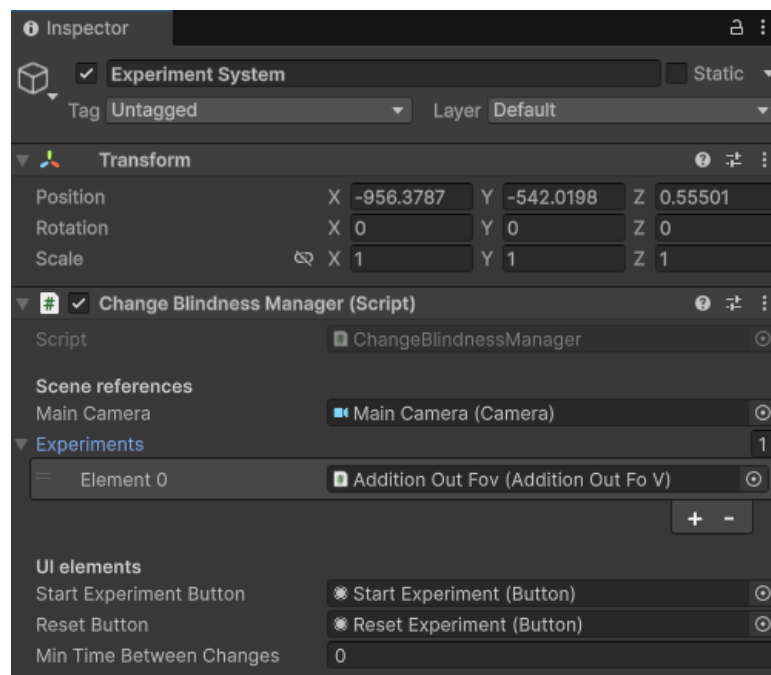


FIGURE 4.1 – Configuration du ChangeBlindnessManager dans l'inspecteur Unity

4.3 Exemples d'expériences disponibles

Pour rappel, ces expériences sont créées sous la forme d'un objet **Unity** auquel est attaché le script de cette dernière.

4.3.1 Expériences InFoV

Les expériences InFoV se déclenchent lorsque l'objet est visible dans le champ de vision et que l'utilisateur réalise une action déclencheuse (porter son regard vers un objet cible en utilisant le cercle de confusion, cligner des yeux, etc.).

ColorInFoV

Change la couleur d'un objet de manière aléatoire lorsque l'utilisateur le regarde directement. Les paramètres configurables sont :

- **Object To Observe** : l'objet qui doit être observé pour déclencher le changement de couleur ;
- **Target Object To Change** : l'objet dont la couleur sera modifiée.

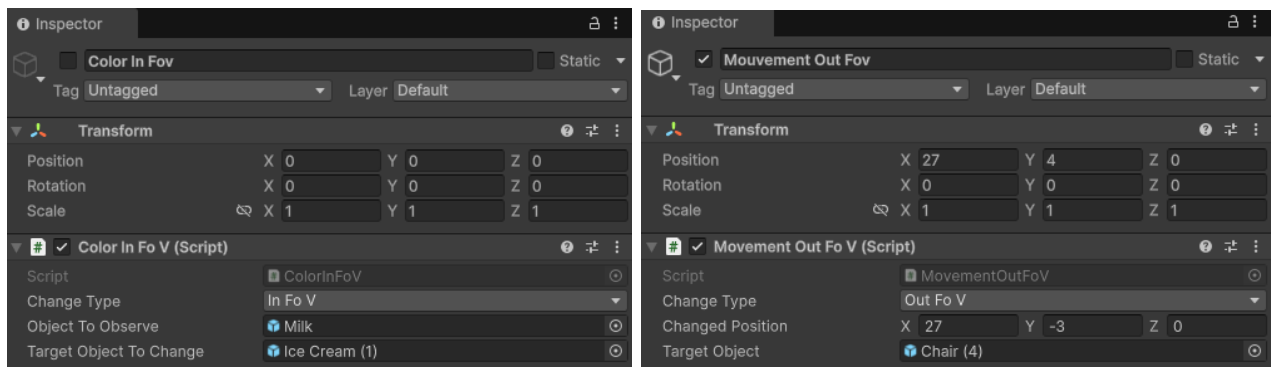
4.3.2 Expériences OutFoV

Les expériences OutFoV se déclenchent lorsque le/les objet(s) cible(s) est/sont en dehors du champ de vision de l'utilisateur.

MovementOutFoV

Déplace un objet vers une nouvelle position lorsqu'il est hors du champ de vision. Les paramètres configurables sont :

- **Changed Position** : la nouvelle position dans l'espace monde où l'objet sera déplacé ;
- **Target Object** : l'objet qui sera déplacé pendant l'expérience.



(a) Color in FoV experiment.

(b) Movement out FoV experiment.

FIGURE 4.2 – Les deux expériences données en exemple vues depuis l'inspecteur Unity.

4.4 Ajout d'expériences personnalisées

4.4.1 Création d'une nouvelle expérience

Pour créer une nouvelle expérience de *change blindness*, il faut :

1. créer une nouvelle classe héritant de **BaseExperiment** ;
2. implémenter les méthodes abstraites requises ;
3. configurer l'objet dans l'inspecteur Unity ;
4. l'ajouter à la liste des expériences du **ChangeBlindnessManager**.

4.4.2 Méthodes à implémenter

Toute expérience personnalisée doit implémenter les méthodes suivantes :

```

public override void ApplyChange()
{
    // Logique pour appliquer le changement
    // Marquer IsChanged = true
}

public override void RevertChange()
{
    // Logique pour annuler le changement
    // Marquer IsChanged = false
}

public override bool IsInFieldOfView(Camera camera)
{
    // Logique pour determiner si l'objet est visible
    // Retourner true/false selon la visibilite
}

```

4.4.3 Exemple d'implémentation

Voici un exemple d'implémentation d'une expérience personnalisée qui modifie la taille d'un objet :

```

public class ScaleChangeInFoV : BaseExperiment
{
    [SerializeField] private GameObject targetObject;
    [SerializeField] private Vector3 newScale = Vector3.one * 2f;
    private Vector3 originalScale;

    private void Start()
    {
        originalScale = targetObject.transform.localScale;
    }

    public override void ApplyChange()
    {
        targetObject.transform.localScale = newScale;
        IsChanged = true;
    }

    public override void RevertChange()
    {
        targetObject.transform.localScale = originalScale;
        IsChanged = false;
    }

    public override bool IsInFieldOfView(Camera camera)
    {
        Vector3 screenPoint =
            camera.WorldToViewportPoint(
                targetObject.transform.position);
    }
}

```

```

    bool isGazeOnTarget =
        IsObjectToObserveInConfusionCircle(targetObject,
                                             camera);

    bool isVisible = screenPoint.z > 0 &&
                    screenPoint.x > 0 && screenPoint.x < 1 &&
                    screenPoint.y > 0 && screenPoint.y < 1;

    return isGazeOnTarget && isVisible;
}
}

```

4.5 Gestion et intégration

4.5.1 Gestion de l'état

Le système maintient un état global qui comprend :

- l'état d'activation de l'expérience (**isExperimentRunning**);
- la liste des objets ayant déjà été modifiés (**activeChanges**);
- le timestamp du dernier changement pour respecter l'intervalle minimum entre les modifications.

4.5.2 Intégration avec l'eye-tracking

Le système s'intègre automatiquement avec les composants d'eye-tracking disponibles dans la scène :

- **GazeManager** : pour obtenir la position du regard et le rayon de précision;
- **BlinkDetector** : pour détecter les clignements d'œil et déclencher les changements pendant ces moments d'occultation naturelle.

5. Système d'illusions visuelles

5.1 Présentation générale

Le système d'illusions visuelles permet de créer et contrôler des effets de post-processing en temps réel dans l'environnement **Unity**, en se basant sur les données de suivi oculaire et de position de tête de l'utilisateur. Ce système implémente diverses illusions perceptuelles qui s'adaptent dynamiquement au comportement visuel de l'utilisateur, créant des expériences immersives et interactives.

Le système supporte plusieurs types d'effets visuels (correspondant à une énumération dans le code) :

- **DepthOfField** : simulation de profondeur de champ photographique basée sur les zones de confusion de regard ;
- **Vignette** : effet de vignettage dynamique qui suit le regard de l'utilisateur ;
- **ChromaticAberration** : aberration chromatique variable selon l'excentricité du regard ;
- **Bloom** : effet de surbrillance adaptatif basé sur la profondeur de regard ;
- **LensDistortion** : distortion de lentille dynamique centrée sur le point de regard ;
- **DirectionalLightRotation** : rotation de l'éclairage directionnel basée sur les mouvements de tête.

5.2 Architecture du système

5.2.1 Composants principaux

Le système d'illusions visuelles est organisé autour de trois composants clés :

- **EffectsManager** : gestionnaire central qui coordonne les effets visuels et gère l'état de l'interface utilisateur ;
- **PostProcessManager** : contrôleur technique des effets de post-processing et de leurs paramètres en temps réel ;
- **IllusionUI** : interface utilisateur pour la sélection et le contrôle des effets visuels.

5.2.2 Configuration de l'EffectsManager

L'**EffectsManager** constitue le point d'entrée principal du système et doit être configuré dans l'inspecteur **Unity** avec les paramètres suivants :

- **Post Process Manager** : référence vers le gestionnaire technique des effets (auto-assigné si sur le même GameObject) ;
- **Effects Panel** : panneau UI contenant les contrôles des effets, masqué automatiquement lors de l'activation des effets.

5.2.3 Configuration du PostProcessManager

Le **PostProcessManager** est le cœur technique du système et nécessite une configuration plus détaillée :

- **Volume** : volume Unity contenant le profil des effets de post-processing ;
- **Main Camera** : caméra principale pour les calculs de profondeur et les effets basés sur la vue ;
- **Gaze Detector** : système de détection du regard fournissant les données d'attention ;
- **Smooth Speed** : facteur de lissage pour les transitions graduelles des effets ;
- **Depth Confusion Zone Analyser** : analyseur fournissant les données de plage de profondeur pour les calculs de mise au point ;
- **Directional Light** : lumière directionnelle principale pour les effets d'éclairage dynamique ;
- paramètres spécifiques aux effets expliqués dans la documentation technique : intensités, seuils, vitesses de transition, etc.

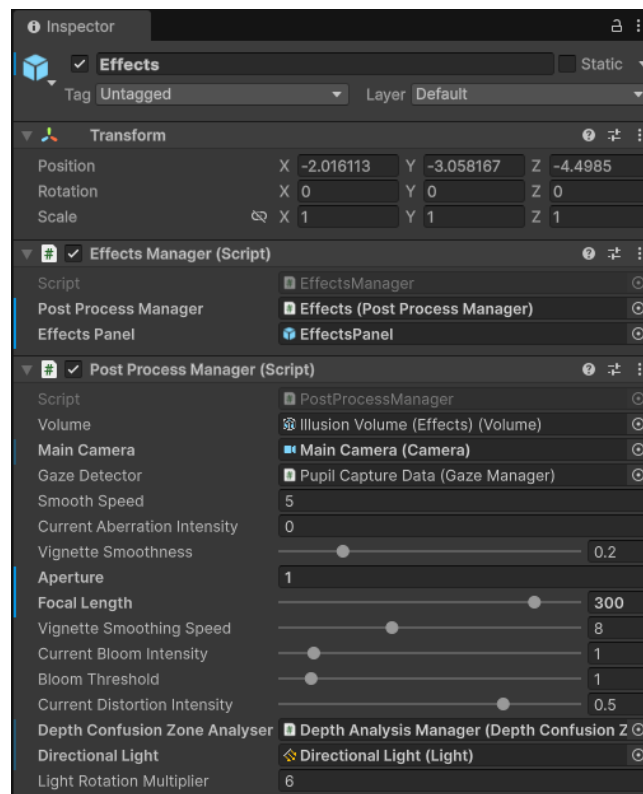


FIGURE 5.1 – Configuration du PostProcessManager dans l'inspecteur Unity

5.3 Effets disponibles

5.3.1 Profondeur de champ (DepthOfField)

Simule une profondeur de champ photographique réaliste en utilisant les principes optiques. L'effet calcule automatiquement :

- la distance de mise au point basée sur la limite proche de la zone de confusion ;
- la distance hyperfocale pour que la limite lointaine tombe dans la zone de netteté acceptable ;
- l'ouverture requise utilisant les contraintes du cercle de confusion.

Pour plus d'informations, il est possible de se reporter à la documentation technique et à l'implémentation elle-même.

5.3.2 Vignettage dynamique (Vignette)

Crée un effet de vignettage dont le centre suit le regard de l'utilisateur. Paramètres configurables :

- `Vignette Intensity` : intensité de l'assombrissement des bords ;
- `Vignette Smoothness` : douceur de la transition des bords ;
- `Vignette Smoothing Speed` : vitesse d'ajustement de la position du vignettage.

5.3.3 Aberration chromatique (ChromaticAberration)

Applique une aberration chromatique variable selon l'excentricité du regard par rapport au centre de l'écran :

- l'intensité est inversement proportionnelle à la proximité du centre ;
- `Current Aberration Intensity` : intensité actuelle appliquée à la simulation de lentille.

5.3.4 Effet de surbrillance (Bloom)

Effet de surbrillance adaptatif basé sur la profondeur du regard :

- `Current Bloom Intensity` : intensité actuelle de l'effet de surbrillance ;
- `Bloom Threshold` : seuil de luminosité au-dessus duquel l'effet s'active.

5.3.5 Distortion de lentille (LensDistortion)

Distortion dynamique qui s'intensifie avec la distance du regard par rapport au centre :

- `Current Distortion Intensity` : intensité actuelle de la distortion ;
- `Distortion Scale` : facteur d'échelle contrôlant la zone de couverture de l'effet.

5.4 Interface utilisateur

5.4.1 Configuration de l'IllusionUI

L'IllusionUI gère les interactions utilisateur et doit être configuré avec :

- `Illusion Manager` : référence vers l'`EffectsManager` pour la délégation des commandes ;
- `Effects Panel` : panneau UI contenant les boutons de contrôle des effets.

Le système masque automatiquement les éléments de visualisation ("Visualization Elements" et "3D Crosshair") lors de l'activation des effets pour une démonstration propre.

5.5 Ajout d'effets personnalisés

5.5.1 Création d'un nouvel effet

Pour créer un nouvel effet de post-processing, il faut :

1. Ajouter le nouvel effet à l'énumération `PostProcessEffect` ;
2. Créer les champs de configuration appropriés dans `PostProcessManager` ;
3. Implémenter la méthode de mise à jour de l'effet ;
4. Ajouter l'effet aux dictionnaires d'initialisation.

5.5.2 Implémentation d'une méthode de mise à jour

Toute méthode de mise à jour d'effet personnalisé doit suivre le pattern suivant :

```
private void UpdateCustomEffect()
{
    if (customEffect == null || gazeDetector == null) return;

    // Calculs bases sur les donnees de regard/tete
    Vector3 gazeWorld = gazeDetector.CurrentGazeHit.point;

    // Application des parametres avec lissage
    float targetValue = CalculateTargetValue(gazeWorld);
    currentValue = Mathf.Lerp(currentValue, targetValue,
                              Time.deltaTime * smoothSpeed);

    // Application a l'effet de post-processing
    customEffect.parameter.value = currentValue;
}
```

5.5.3 Intégration dans le système

L'effet doit être ajouté aux dictionnaires situés dans la méthode `InitializeEffectDictionaries()` de la classe `PostProcessManager`. Il est nécessaire de l'ajouter au premier dictionnaire `effectsComponents` si c'est un effet intégré à Unity et qu'il possède son entrée dans le volume global de la scène.

6. Analyseur de zones de regard (GazeAnalyzer)

6.1 Présentation générale

Le système GazeAnalyzer permet d'analyser quantitativement la distribution du regard de l'utilisateur sur les objets présents dans la scène virtuelle. Il détermine quelle proportion de la zone de confusion du regard est occupée par chaque objet analysable.

Le système fonctionne en deux phases principales : une phase de segmentation par rendu d'identifiants colorés et une phase d'analyse proportionnelle des pixels dans la zone de regard circulaire. L'ensemble utilise un traitement asynchrone sur GPU et des threads en arrière-plan pour l'analyse.

Étant donné le passage à une représentation en trois dimensions de la zone de regard via la *search light*, le système n'est pas utilisé dans le projet, mais l'objet **Unity** l'englobant est toujours présent dans l'inspecteur.

6.2 Architecture du système

6.2.1 Composants principaux

Le système GazeAnalyzer est composé de deux scripts complémentaires :

- **GazeZoneAnalyzer** : analyseur technique principal qui effectue la segmentation des objets et le calcul des proportions ;
- **GazeProportionAnalyzer** : analyseur périodique qui interroge le système et rapporte les résultats significatifs.

6.2.2 Configuration du GazeZoneAnalyzer

Le **GazeZoneAnalyzer** constitue le cœur technique du système et nécessite la configuration suivante dans l'inspecteur **Unity** :

- **Gaze Detector** : référence vers le système de détection du regard (**GazeManager**) ;
- **Main Camera** : caméra principale utilisée comme référence pour la caméra de segmentation ;
- **Parent Hierarchy** : objet parent depuis lequel peupler automatiquement la liste des objets analysables ;
- **Analyzable Objects** : liste des **GameObjects** à analyser (peuplée automatiquement depuis la hiérarchie parent). Les objets formant l'arrière plan sont distingués dans le script par la présence de *Background* dans leurs noms.

6.2.3 Configuration du GazeProportionAnalyzer

Le GazeProportionAnalyzer gère l'analyse périodique et les rapports :

- Gaze Zone Analyzer : référence vers le GazeZoneAnalyzer fournissant les proportions par objet ;
- Analysis Interval : intervalle de temps (en secondes) entre les analyses consécutives ;
- Minimum Proportion Threshold : proportion minimale (0-1) requise pour considérer un objet comme significatif.

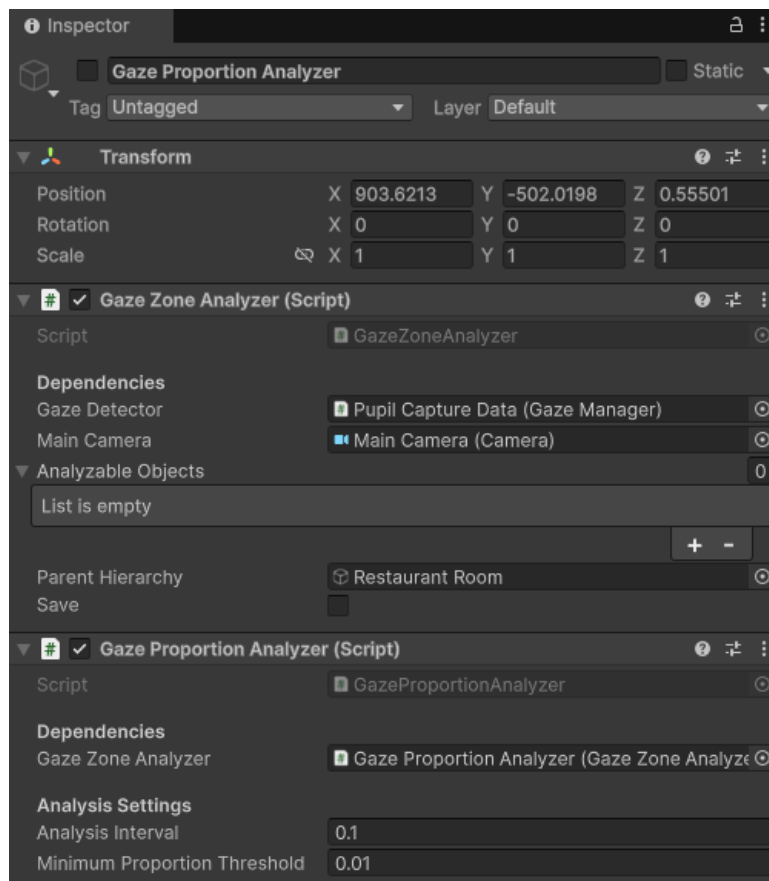


FIGURE 6.1 – Configuration du du GazeAnalyzer dans l'inspecteur Unity

6.3 Fonctionnement technique

6.3.1 Principe de segmentation par identifiants colorés

Le système utilise une technique de segmentation avancée où chaque objet analysable se voit attribuer un identifiant unique encodé sous forme de couleur RGB. Une caméra de segmentation dédiée effectue un rendu spécialisé où chaque objet apparaît dans sa couleur d'identifiant, permettant une identification précise au niveau du pixel.

6.3.2 Analyse de la zone de regard

L'analyse se concentre sur la zone circulaire de confusion du regard, définie par le système de suivi oculaire. Le système compte les pixels de chaque couleur d'identifiant présents dans ce cercle et calcule la proportion relative occupée par chaque objet.

6.3.3 Traitement asynchrone

Pour maintenir les performances, le système utilise :

- Lecture asynchrone GPU (`AsyncGPUReadback`) pour récupérer les données de pixels ;
- Traitement en thread séparé pour l'analyse des pixels ;
- Mise en cache des résultats pour éviter les blocages pendant le traitement.

6.3.4 Interprétation des résultats

Les résultats sont fournis sous forme de dictionnaire associant chaque `GameObject` à sa proportion (valeur entre 0 et 1) de la zone de regard. Une proportion de 0.25 signifie que l'objet occupe 25% de la zone de confusion du regard.

7. Système de search light

7.1 Présentation générale

Le système de search light nous permet de modéliser le problème de la zone de regard de façon à récupérer la profondeur minimum et maximum de la zone du regard avec un compute shader en premier lieu. Ce dernier nous permet de faire ça à l'aide du depth buffer envoyé par une camera secondaire. Aussi, ce système nous permet de récupérer les fragments contenus dans la zone de regard, ce qui nous permet de modifier l'aspect visuel de ces fragments à notre guise.

7.2 Changement des propriétés des matériaux

Afin de modifier les propriétés des matériaux des objets qui nous intéressent, nous avons mis en place un système de shaders. Ce système est crée sur **Shader Graph**, qui nous permet de créer un vertex et un fragment shader sans difficulté et de modifier les entrées de ces dernier afin de changer les propriétés des fragments contenus dans la zone du regard. Actuellement, la seule propriété qu'on change est l'émissivité du matériau. Cependant, pour des raisons de simplicité il faut créer et sélectionner des matériaux utilisant ce shader sur chaque object sur lequel on veut appliquer l'effet pour avoir une visualisation satisfaisante. Le shader en question est dans le dossier **assets/Resources/Shaders** nommé **ProximityHighlightShader**. De ce fait, on peut créer de multiples shaders pour chaque effet en modifiant les propriétés des matériaux utilisés.

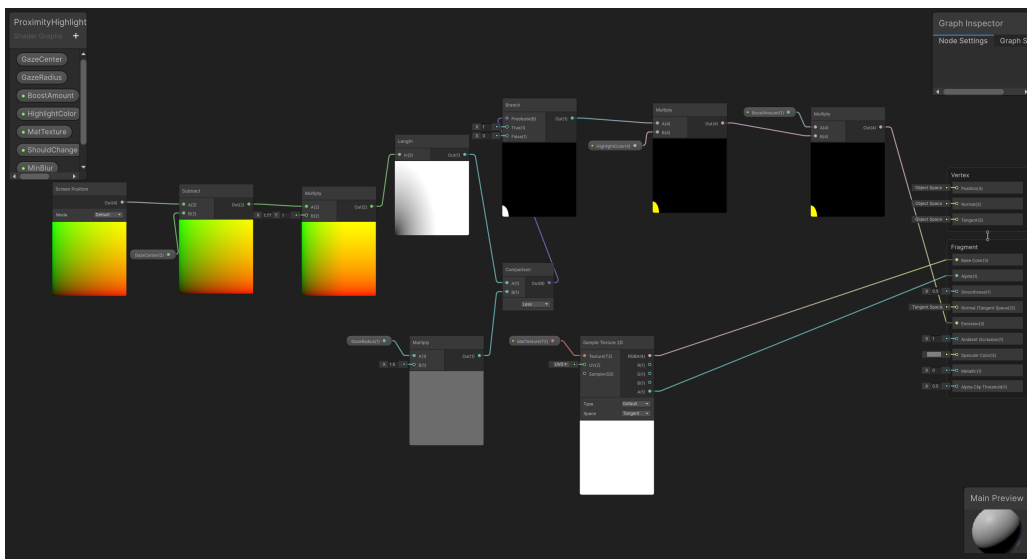


FIGURE 7.1 – Schéma du Lit Shader Graph utilisé pour avoir l'effet de search light sur les objets

7.3 Profondeur minimum et maximum

Pour certains effets, dont la profondeur de champ, l'information de la profondeur est crucial pour avoir un effet convaincant. Pour ce faire, un compute shader a été créé afin de calculer la profondeur minimum et maximum de la zone du regard utilisant le depth buffer d'une seconde camera nommée **depth camera**. Cette camera suivra la camera principale mais ne fera le rendu que des objets qui nous intéressent pour le calcul de la profondeur, donc tous sauf les murs, le plafond et le sol. Pour configurer cela, il faut mettre les objets d'arrière plan dans une couche de rendu dédiée (la couche **background** dans notre scène), et configurer **depth camera** pour que le culling mask prend toutes les couches sauf la couche des objets en arrière plan.

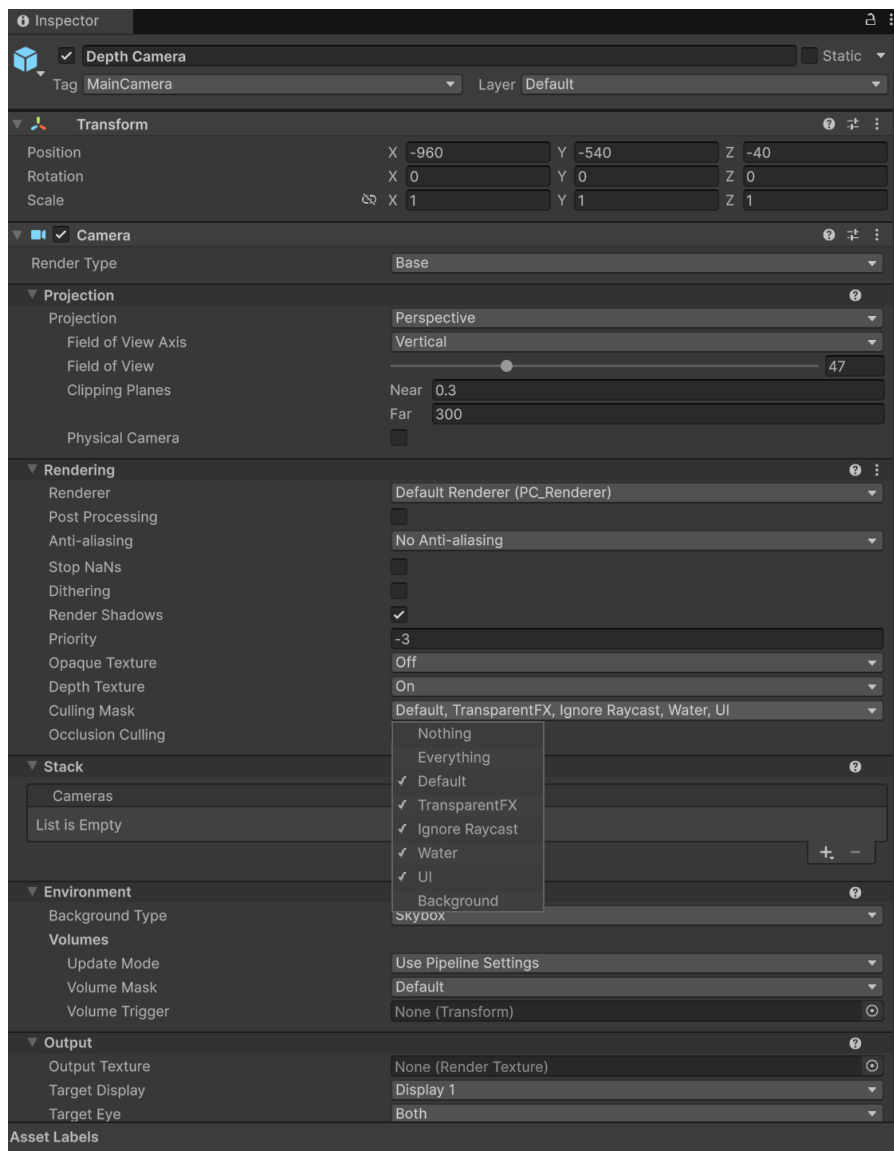


FIGURE 7.2 – Configuration du culling mask de la camera secondaire

Le **GameObject Depth Analysis Manager** est celui qui assure la gestion du système d'analyse de profondeur. Le compute shader associé est situé dans **assets/Resources/Shaders** nommé **Zbuffer**.

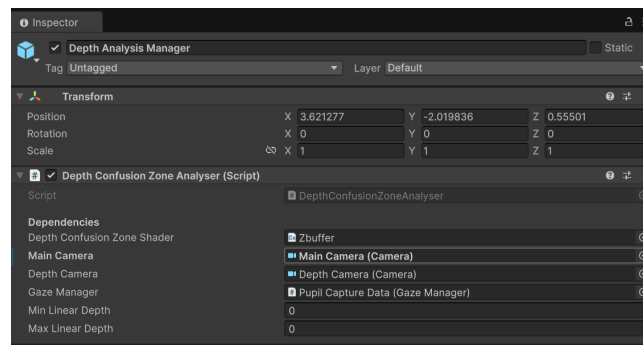


FIGURE 7.3 – Configuration de Depth Analysis Manager

Bibliographie

- [1] Daniel Martin, Xin Sun, Diego Gutierrez, and Belen Masia. A study of change blindness in immersive environments. *IEEE Transactions on Visualization and Computer Graphics*, 29(5) :2446–2455, 2023.